

Cours Complet - Thèse DNN

COURS COMPLET : Attaques d'Extraction et Défenses pour les DNN
au-delà de ReLU

TABLE DES MATIÈRES

PARTIE I : FONDAMENTAUX DU DEEP LEARNING

Chapitre 1 : Le Perceptron

- 1.1 Introduction Historique
- 1.2 Modèle Mathématique
- 1.3 Fonction d'Activation Originale
- 1.4 Interprétation Géométrique
- 1.5 Algorithme d'Apprentissage
- 1.6 Théorème de Convergence
- 1.7 Limitation : Le Problème XOR
- 1.8 Code Complet : Perceptron

Chapitre 2 : Réseaux Multicouches (MLP)

- 2.1 Motivation
- 2.2 Architecture
- 2.3 Notation Mathématique
- 2.4 Exemple : MLP 2-2-1 pour XOR
- 2.5 Théorème d'Approximation Universelle
- 2.6 Propagation Avant (Forward Pass)
- 2.7 Pourquoi c'est Important pour l'Extraction

Chapitre 3 : Rétropropagation (Backpropagation)

- 3.1 Le Problème de l'Apprentissage
- 3.2 La Règle de la Chaîne
- 3.3 Algorithme de Backpropagation
- 3.4 Dérivation Détaillée
- 3.5 Lien avec les Attaques d'Extraction
- 3.6 Code Complet : MLP avec Backpropagation

Chapitre 4 : Fonctions d'Activation

- 4.1 Rôle des Fonctions d'Activation
- 4.2 Propriétés Souhaitables
- 4.3 Sigmoid
- 4.4 Tanh
- 4.5 ReLU (Rectified Linear Unit)
- 4.6 Propriété Cruciale de ReLU : Piecewise Linear
- 4.7 GELU (Gaussian Error Linear Unit)
- 4.8 SiLU / Swish
- 4.9 Comparaison Critique pour ta Thèse
- 4.10 Code : Implémentation et Visualisation

Chapitre 5 : Réseaux Convolutionnels (CNN)

- 5.1 Motivation
- 5.2 L'Opération de Convolution
- 5.3 Paramètres de Convolution
- 5.4 Architecture Typique
- 5.5 Pooling
- 5.6 Partage de Poids
- 5.7 Code : CNN Simple
- 5.8 Implications pour l'Extraction

Chapitre 6 : Architectures Modernes

- 6.1 ResNet et Skip Connections
- 6.2 Transformers et Attention
- 6.3 Pourquoi GELU dans les Transformers ?
- 6.4 Code : Bloc Transformer Simplifié

PARTIE II : CRYPTOGRAPHIE ET SÉCURITÉ ML

Chapitre 7 : Fondamentaux de Cryptographie

- 7.1 Concepts de Base
- 7.2 Cryptanalyse Différentielle
- 7.3 Analogie avec les DNN

Chapitre 8 : Modèles d'Attaque en ML

- 8.1 Niveaux d'Accès
- 8.2 Types d'Attaques
- 8.3 Model Extraction : Approches
- 8.4 Code : Oracle et Attaque Simple

Chapitre 9 : Model Stealing et Extraction

- 9.1 Historique des Attaques
- 9.2 Différence Fondamentale
- 9.3 Métriques

PARTIE III : ATTAQUES D'EXTRACTION CRYPTANALYTIQUES

Chapitre 10 : CRYPTO 2020 - L'Attaque Fondatrice

- 10.1 Paper
- 10.2 Idée Principale
- 10.3 Points Critiques
- 10.4 Algorithme Simplifié
- 10.5 Complexité
- 10.6 Code : Détection de Points Critiques

Chapitre 11 : EUROCRYPT 2024 - Temps Polynomial

- 11.1 Paper
- 11.2 Amélioration Principale
- 11.3 Complexité Améliorée
- 11.4 Techniques Clés

Chapitre 12 : EUROCRYPT 2025 - Hard-Label Setting

- 12.1 Paper
- 12.2 Défi
- 12.3 Approche
- 12.4 Complexité

PARTIE IV : AU-DELÀ DE ReLU - LE CŒUR DE LA THÈSE

Chapitre 13 : Pourquoi GELU/SiLU Résistent

- 13.1 Le Problème Fondamental
- 13.2 Analyse Mathématique
- 13.3 Conséquences pour l'Extraction
- 13.4 Code : Démonstration

Chapitre 14 : Pistes de Recherche

- 14.1 Question Centrale de ta Thèse
- 14.2 Piste 1 : Approximation Piecewise-Linear
- 14.3 Piste 2 : Points d'Inflexion
- 14.4 Piste 3 : Méthodes d'Optimisation
- 14.5 Piste 4 : Attaques Hybrides
- 14.6 Piste 5 : Nouvelles Signatures
- 14.7 Code : Exploration des Points d'Inflexion

Chapitre 15 : Défenses et Contre-mesures

- 15.1 Types de Défenses
- 15.2 Perturbation des Sorties
- 15.3 Rate Limiting
- 15.4 Watermarking
- 15.5 MEA-Defender (Défense Avancée)
- 15.6 Code : Défenses Simples

PARTIE V : EXERCICES ET CODE

- Exercice Complet 1 : Perceptron et MLP
- Exercice Complet 2 : Fonctions d'Activation
- Exercice Complet 3 : Détection de Points Critiques
- Exercice Complet 4 : Attaque de Model Stealing

RÉFÉRENCES BIBLIOGRAPHIQUES

- Papiers Fondamentaux
- Livres
- Ressources en Ligne

CONCLUSION

COURS COMPLET : Attaques d'Extraction et Défenses pour les DNN au-delà de ReLU

Thèse de Doctorat Doctorant : Tidiane DIALLO **Directeur :** Pr.
Abdoul Aziz Ciss (EPT) **Laboratoire :** CRISIN'2D / Équipe LTISI
Période : 2025-2028

TABLE DES MATIÈRES

1. PARTIE I : FONDAMENTAUX DU DEEP LEARNING
 - Chapitre 1 : Le Perceptron
 - Chapitre 2 : Réseaux Multicouches (MLP)
 - Chapitre 3 : Rétropropagation
 - Chapitre 4 : Fonctions d'Activation
 - Chapitre 5 : Réseaux Convolutionnels (CNN)
 - Chapitre 6 : Architectures Modernes
2. PARTIE II : CRYPTOGRAPHIE ET SÉCURITÉ ML

- Chapitre 7 : Fondamentaux de Cryptographie
 - Chapitre 8 : Modèles d'Attaque en ML
 - Chapitre 9 : Model Stealing et Extraction
 - 3. PARTIE III : ATTAQUES D'EXTRACTION CRYPTANALYTIQUES
 - Chapitre 10 : CRYPTO 2020 - L'Attaque Fondatrice
 - Chapitre 11 : EUROCRYPT 2024 - Temps Polynomial
 - Chapitre 12 : EUROCRYPT 2025 - Hard-Label Setting
 - 4. PARTIE IV : AU-DELÀ DE ReLU - LE CŒUR DE LA THÈSE
 - Chapitre 13 : Pourquoi GELU/SiLU Résistent
 - Chapitre 14 : Pistes de Recherche
 - Chapitre 15 : Défenses et Contre-mesures
 - 5. PARTIE V : EXERCICES ET CODE
-

PARTIE I : FONDAMENTAUX DU DEEP LEARNING

Chapitre 1 : Le Perceptron

1.1 Introduction Historique

Le perceptron, inventé par **Frank Rosenblatt en 1958**, est le premier modèle de neurone artificiel capable d'apprendre. C'est la brique fondamentale de tout réseau de neurones.

1.2 Modèle Mathématique

Un perceptron calcule une **combinaison linéaire** des entrées, puis applique une **fonction d'activation** :

Entrées: $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$

Poids: $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$

Biais: b

Pré-activation: $z = \sum_i w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$

Sortie: $y = f(z)$

Notation matricielle :

$$z = \mathbf{w}^T \mathbf{x} + b$$

$$y = f(z)$$

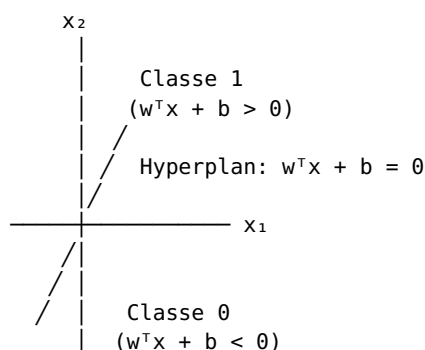
1.3 Fonction d'Activation Originale

Le perceptron original utilise la **fonction de Heaviside** (step function) :

$$f(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$

1.4 Interprétation Géométrique

Le perceptron définit un **hyperplan** qui sépare l'espace en deux :



Point crucial pour ta thèse : Cette séparation linéaire est la BASE des régions créées par ReLU !

1.5 Algorithme d'Apprentissage

```
# Règle du perceptron
pour chaque exemple (x, y_vrai):
    y_pred = 1 si (wTx + b) ≥ 0 sinon 0
    erreur = y_vrai - y_pred

# Mise à jour si erreur ≠ 0
w ← w + η × erreur × x
b ← b + η × erreur
```

où η est le **taux d'apprentissage** (learning rate).

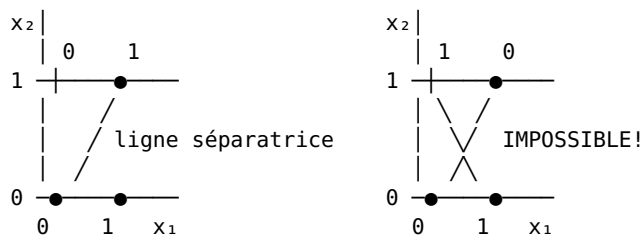
1.6 Théorème de Convergence

Théorème (Novikoff, 1962) : Si les données sont **linéairement séparables**, l'algorithme du perceptron converge en un nombre fini d'itérations.

1.7 Limitation : Le Problème XOR

Le perceptron ne peut résoudre que les problèmes **linéairement séparables**.

Fonction AND (séparable) Fonction XOR (NON séparable)



● = classe 1, ○ = classe 0

Solution : Utiliser plusieurs couches → **MLP** !

1.8 Code Complet : Perceptron

```
import numpy as np

class Perceptron:
    """
    Perceptron simple pour classification binaire.
    """

    def __init__(self, n_features, learning_rate=0.1):
        """
        Args:
            n_features: Nombre de features d'entrée
            learning_rate: Taux d'apprentissage  $\eta$ 
        """
        self.weights = np.random.randn(n_features) * 0.01
        self.bias = 0.0
        self.lr = learning_rate

    def activation(self, z):
        """Fonction de Heaviside"""
        return 1 if z >= 0 else 0

    def predict(self, x):
        """Calcule la sortie pour une entrée x"""
        z = np.dot(self.weights, x) + self.bias
        return self.activation(z)

    def train(self, X, y, epochs=100):
        """
        Entraîne le perceptron.

        Args:
            X: Données d'entrée (n_samples, n_features)
        """
```

y: Labels (n_samples,)
epochs: Nombre d'époques

Returns:

Liste des erreurs par époque

"""

errors = []

for epoch **in** range(epochs):
 total_errors = 0

for xi, yi **in** zip(X, y):
 prediction = self.predict(xi)
 error = yi - prediction

Mise à jour des poids

self.weights += self.lr * error * xi
 self.bias += self.lr * error

total_errors += abs(error)

errors.append(total_errors)

if total_errors == 0:
 print(f"Convergence à l'époque {epoch + 1}")
break

return errors

===== EXEMPLE : FONCTION AND =====

X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

y_and = np.array([0, 0, 0, 1])

perceptron = Perceptron(n_features=2)
 perceptron.train(X_and, y_and)

print("Résultats AND:")

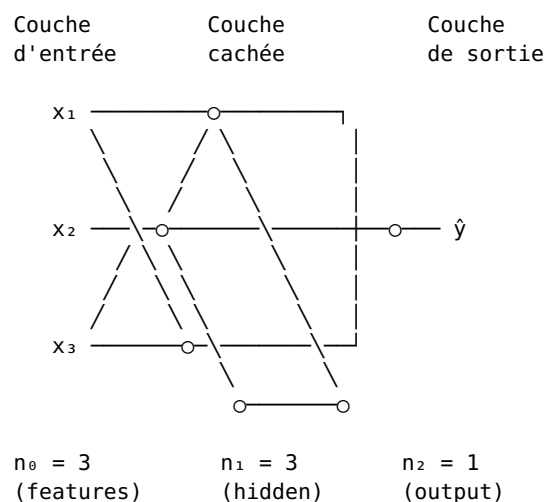
for xi, yi **in** zip(X_and, y_and):
 print(f" {xi} -> {perceptron.predict(xi)} (attendu: {yi})")

Chapitre 2 : Réseaux Multicouches (MLP)

2.1 Motivation

Le perceptron simple ne peut pas résoudre XOR car il n'est pas linéairement séparable. La solution : **empiler plusieurs couches** de neurones.

2.2 Architecture



2.3 Notation Mathématique

Pour un réseau à L couches :

Couche l (pour $l = 1, \dots, L$) :

Pré-activation: $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$

Activation: $a^{(l)} = f^{(l)}(z^{(l)})$

où : - $W^{(l)}$: matrice de poids de dimension $(n_l \times n_{l-1})$ - $b^{(l)}$: vecteur de biais de dimension $(n_l \times 1)$ - $f^{(l)}$: fonction d'activation de la couche l - $a^{(0)} = x$: l'entrée du réseau

2.4 Exemple : MLP 2-2-1 pour XOR

Architecture:

- Entrée: 2 neurones (x_1, x_2)
- Couche cachée: 2 neurones (sigmoid)
- Sortie: 1 neurone (sigmoid)

Paramètres:

- $W^{(1)}$: matrice 2×2
- $b^{(1)}$: vecteur 2×1
- $W^{(2)}$: matrice 1×2
- $b^{(2)}$: scalaire

Total: $2 \times 2 + 2 + 1 \times 2 + 1 = 9$ paramètres

2.5 Théorème d'Approximation Universelle

Théorème (Cybenko, 1989; Hornik, 1991) :

Un MLP avec une seule couche cachée contenant suffisamment de neurones peut approximer n'importe quelle fonction continue sur un compact à n'importe quelle précision.

Interprétation : Les MLP sont des **approximateurs universels** !

En pratique, les réseaux **profonds** (plusieurs couches) sont plus efficaces : - Moins de paramètres pour la même expressivité - Meilleure généralisation - Représentations hiérarchiques

2.6 Propagation Avant (Forward Pass)

```
def forward_pass(x, weights, biases, activations):  
    """  
    Calcule la sortie du réseau.  
  
    Args:  
        x: entrée  
        weights: liste des matrices  $W^{(l)}$   
        biases: liste des vecteurs  $b^{(l)}$   
        activations: liste des fonctions  $f^{(l)}$   
  
    Returns:  
        sortie du réseau  
    """  
    a = x #  $a^{(0)} = x$   
  
    for l in range(len(weights)):  
        z = weights[l] @ a + biases[l] #  $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$   
        a = activations[l](z) #  $a^{(l)} = f^{(l)}(z^{(l)})$   
  
    return a
```

2.7 Pourquoi c'est Important pour l'Extraction

Lors d'une attaque d'extraction :

1. **L'attaquant envoie** des entrées x au modèle (via API)
2. **Il observe** les sorties $\hat{y}$
3. **Il cherche à reconstruire** les $W^{(l)}$ et $b^{(l)}$

Le forward pass est ce que l'attaquant peut **observer** !

Chapitre 3 : Rétropropagation (Backpropagation)

3.1 Le Problème de l'Apprentissage

Objectif : Trouver les poids W et biais b qui minimisent une fonction de perte L .

Fonction de perte typiques : - MSE (régression) : $L = \frac{1}{2}(\hat{y} - y)^2$ - Cross-entropy (classification) : $L = -\sum y_i \log(\hat{y}_i)$

Méthode : Descente de gradient

$$\theta \leftarrow \theta - \eta \times \partial L / \partial \theta$$

où θ représente tous les paramètres (W, b).

3.2 La Règle de la Chaîne

Si $y = f(g(x))$, alors :

$$dy/dx = dy/dg \times dg/dx$$

Pour un réseau de neurones, on “déroule” la chaîne depuis la perte jusqu’aux poids :

$$\partial L / \partial W^{(l)} = \partial L / \partial a^{(L)} \times \partial a^{(L)} / \partial z^{(L)} \times \dots \times \partial z^{(l)} / \partial W^{(l)}$$

3.3 Algorithme de Backpropagation

ALGORITHME BACKPROPAGATION
=====

1. FORWARD PASS

Pour $l = 1, \dots, L$:
 $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$
 $a^{(l)} = f^{(l)}(z^{(l)})$

Calculer la perte: $L = \text{Loss}(a^{(L)}, y)$

2. BACKWARD PASS

Couche de sortie
 $\delta^{(L)} = \partial L / \partial z^{(L)} = \partial L / \partial a^{(L)} \odot f'^{(L)}(z^{(L)})$

Couches cachées (de $L-1$ à 1)
Pour $l = L-1, \dots, 1$:
 $\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'^{(l)}(z^{(l)})$

3. CALCUL DES GRADIENTS

Pour $l = 1, \dots, L$:
 $\partial L / \partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$
 $\partial L / \partial b^{(l)} = \delta^{(l)}$

4. MISE À JOUR DES POIDS

Pour $l = 1, \dots, L$:
 $W^{(l)} \leftarrow W^{(l)} - \eta \times \partial L / \partial W^{(l)}$
 $b^{(l)} \leftarrow b^{(l)} - \eta \times \partial L / \partial b^{(l)}$

où $\odot$ désigne le produit élément par élément (Hadamard).

3.4 Dérivation Détaillée

Couche de sortie (L) :

Pour MSE avec sortie $a^{(L)} = z^{(L)}$ (pas d'activation) :

$$L = \frac{1}{2} (a^{(L)} - y)^2$$
$$\partial L / \partial a^{(L)} = a^{(L)} - y$$
$$\delta^{(L)} = a^{(L)} - y$$

Couche cachée ($l < L$) :

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'^{(l)}(z^{(l)})$$

Gradients :

$$\partial L / \partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T \quad (\text{produit extérieur})$$
$$\partial L / \partial b^{(l)} = \delta^{(l)}$$

3.5 Lien avec les Attaques d'Extraction

Observation clé (Carlini et al., CRYPTO 2020) :

Un attaquant peut estimer les gradients par **différences finies** :

$$\partial f / \partial x_i \approx [f(x + \epsilon e_i) - f(x - \epsilon e_i)] / (2\epsilon)$$

où e_i est le vecteur unitaire dans la direction i .

Ces gradients **révèlent de l'information** sur la structure interne du réseau !

3.6 Code Complet : MLP avec Backpropagation

```
import numpy as np

def sigmoid(x):
    """Fonction sigmoid  $\sigma(x) = 1/(1+e^{-x})$ """
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_derivative(x):
    """ $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ """
    s = sigmoid(x)
    return s * (1 - s)

class MLP:
    """
    Multi-Layer Perceptron avec backpropagation.
    """

    def __init__(self, layer_sizes):
        """
        Args:
            layer_sizes: liste [n_input, n_hidden1, ..., n_output]
        """
        self.L = len(layer_sizes) - 1 # nombre de couches (hors
entrée)

        # Initialisation Xavier
        self.W = []
        self.b = []
        for l in range(self.L):
            n_in, n_out = layer_sizes[l], layer_sizes[l+1]
            self.W.append(np.random.randn(n_out, n_in) * np.sqrt(2.0
/ n_in))

            self.b.append(np.zeros((n_out, 1)))

        # Cache pour backprop
        self.cache = {}

    def forward(self, X):
        """
        Forward pass.

        Args:
            X: entrées de shape (n_features, n_samples)

        Returns:
            Sortie du réseau
        """
        self.cache['A0'] = X
        A = X

        for l in range(self.L):
            Z = self.W[l] @ A + self.b[l]
            A = sigmoid(Z)
            self.cache[f'Z{l+1}'] = Z
            self.cache[f'A{l+1}'] = A
```

```

        return A

def backward(self, Y, learning_rate=0.1):
    """
    Backward pass avec mise à jour des poids.

    Args:
        Y: labels de shape (n_output, n_samples)
        learning_rate: taux d'apprentissage
    """
    m = Y.shape[1] # nombre d'exemples

    # Couche de sortie
    A_L = self.cache[f'A{self.L}']
    Z_L = self.cache[f'Z{self.L}']
    dA = A_L - Y
    dZ = dA * sigmoid_derivative(Z_L)

    # Backward à travers les couches
    for l in range(self.L, 0, -1):
        A_prev = self.cache[f'A{l-1}']

        # Gradients
        dW = (1/m) * dZ @ A_prev.T
        db = (1/m) * np.sum(dZ, axis=1, keepdims=True)

        # Propager le gradient à la couche précédente
        if l > 1:
            Z_prev = self.cache[f'Z{l-1}']
            dZ = (self.W[l-1].T @ dZ) * sigmoid_derivative(Z_prev)

        # Mise à jour des poids
        self.W[l-1] -= learning_rate * dW
        self.b[l-1] -= learning_rate * db

def train(self, X, Y, epochs=10000, lr=0.5, print_every=1000):
    """Entraînement du réseau."""
    losses = []

    for epoch in range(epochs):
        # Forward
        output = self.forward(X)

        # Perte MSE
        loss = np.mean((output - Y) ** 2)
        losses.append(loss)

        # Backward
        self.backward(Y, lr)

        if epoch % print_every == 0:
            print(f"Epoch {epoch}: Loss = {loss:.6f}")

    return losses

def predict(self, X):
    """Prédiction avec seuil 0.5"""
    return (self.forward(X) > 0.5).astype(int)

# ===== EXEMPLE : FONCTION XOR =====
X_xor = np.array([[0, 0, 1, 1],
                  [0, 1, 0, 1]])
Y_xor = np.array([[0, 1, 1, 0]])

mlp = MLP([2, 4, 1]) # 2 entrées, 4 neurones cachés, 1 sortie
mlp.train(X_xor, Y_xor, epochs=10000, lr=2.0, print_every=2000)

print("\nRésultats XOR:")
output = mlp.forward(X_xor)
for i in range(4):
    print(f" [{X_xor[0,i]}, {X_xor[1,i]}] -> {output[0,i]:.4f} ≈ {int(output[0,i] > 0.5)}")

```

Chapitre 4 : Fonctions d'Activation

4.1 Rôle des Fonctions d'Activation

Les fonctions d'activation introduisent la **non-linéarité** :

Sans activation: $y = W_2(W_1x) = (W_2W_1)x = Wx$
→ Équivalent à une seule couche linéaire !

Avec activation: $y = W_2 \times f(W_1x)$
→ Non-linéaire, peut apprendre des fonctions complexes

4.2 Propriétés Souhaitables

Propriété	Description	Importance
Non-linéarité	$f(ax+by) \neq af(x)+bf(y)$	Expressivité
Dérivable	$f'(x)$ existe	Backpropagation
Non-saturante	$f'(x)$ ne s'annule pas	Évite vanishing gradient
Efficace	Calcul rapide	Performance

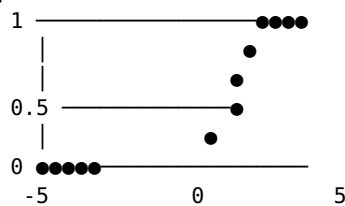
4.3 Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

$$\text{Dérivée: } \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Sortie: (0, 1)

Graphe:



Avantages : - Sortie interprétable comme probabilité - Bornée entre 0 et 1

Inconvénients : - **Saturation** : gradient proche de 0 pour $|x|$ grand → vanishing gradient - Pas centrée autour de 0

4.4 Tanh

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x}) = 2\sigma(2x) - 1$$

$$\text{Dérivée: } \tanh'(x) = 1 - \tanh^2(x)$$

Sortie: (-1, 1)

Avantage sur sigmoid : Centrée autour de 0 → meilleure convergence

Inconvénient : Saturation toujours présente

4.5 ReLU (Rectified Linear Unit)

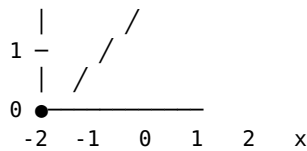
$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

$$\text{Dérivée: } \text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x < 0 \end{cases}$$

(non définie en $x=0$)

Graphe:





Avantages : - Calcul très rapide - Pas de saturation pour $x > 0$ - Activations sparse (beaucoup de 0)

Inconvénients : - **Dying ReLU** : si $z < 0$ toujours, le neurone “meurt” - Pas centrée autour de 0

4.6 Propriété Cruciale de ReLU : Piecewise Linear

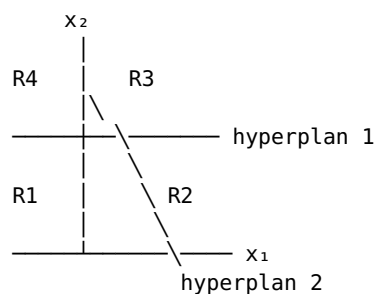
C'EST LE POINT CENTRAL DE TA THÈSE !

ReLU crée des **régions linéaires par morceaux** :

Pour un réseau avec n neurones ReLU:

1. Chaque neurone définit un hyperplan: $w^T x + b = 0$
2. L'espace est découpé en régions où chaque neurone est soit actif (x) soit inactif (0)
3. Dans chaque région, le réseau est une fonction LINÉAIRE
4. Nombre max de régions: $0(2^n)$ pour n neurones

Visualisation 2D avec 3 neurones :



Chaque région R_i = fonction linéaire différente !

Pourquoi c'est important pour l'extraction ?

Les **points critiques** (intersections des hyperplans) révèlent les poids du réseau !

4.7 GELU (Gaussian Error Linear Unit)

Paper : Hendrycks & Gimpel (2016)

$$\text{GELU}(x) = x \times \Phi(x)$$

$$\begin{aligned} \text{où } \Phi(x) &= \text{CDF de la loi normale standard} \\ &= \frac{1}{2} [1 + \text{erf}(x/\sqrt{2})] \end{aligned}$$

$$\text{Dérivée: } \text{GELU}'(x) = \Phi(x) + x \times \phi(x)$$

$$\text{où } \phi(x) = \text{PDF normale} = (1/\sqrt{2\pi}) \times e^{(-x^2/2)}$$

Approximation (utilisée en pratique) :

$$\text{GELU}(x) \approx 0.5x \times [1 + \tanh(\sqrt{2/\pi} \times (x + 0.044715x^3))]$$

Graphes :



-0.2| zone négative!
 -2 -1 0 1 2 x

Propriétés : - Lisse (dérivable partout) - Non monotone (petite zone négative autour de $x \approx -0.17$) - Utilisée dans **BERT, GPT, ViT**

Pourquoi plus résistante aux attaques ? > Pas de “points critiques” nets car la transition est **continue** !

4.8 SiLU / Swish

Paper : Ramachandran et al. (2017) - “Searching for Activation Functions”

$$\text{SiLU}(x) = x \times \sigma(x) = x / (1 + e^{-x})$$

$$\text{Dérivée: SiLU}'(x) = \sigma(x) \times [1 + x(1 - \sigma(x))]$$

Propriétés : - Très similaire à GELU - **Lisse** partout - Utilisée dans **EfficientNet, YOLOv5/v8**

4.9 Comparaison Critique pour ta Thèse

Activation	Type	Vulnérable ?	Utilisée dans
ReLU	Piecewise linear	OUI	CNN classiques
Leaky ReLU	Piecewise linear	OUI	-
GELU	Lisse	NON	Transformers (BERT, GPT)
SiLU	Lisse	NON	Vision (EfficientNet, YOLO)

Tableau des dérivées :

Activation	f'(x)	Points critiques ?
ReLU	0 ou 1 (discret)	OUI (détectables)
GELU	Continue de 0 à ~1.08	NON
SiLU	Continue de 0 à ~1.1	NON

4.10 Code : Implémentation et Visualisation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erf

# ===== DÉFINITIONS =====

def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def gelu(x):
    """GELU exact"""
    return 0.5 * x * (1 + erf(x / np.sqrt(2)))

def gelu_derivative(x):
    """GELU'(x) = Φ(x) + x×φ(x)"""
    phi_cdf = 0.5 * (1 + erf(x / np.sqrt(2)))
    phi_pdf = np.exp(-0.5 * x**2) / np.sqrt(2 * np.pi)
    return phi_cdf + x * phi_pdf

def silu(x):
    """SiLU = x × σ(x)"""
```

```

        return x * sigmoid(x)

def silu_derivative(x):
    """SiLU'(x) =  $\sigma(x) \times [1 + x(1-\sigma(x))]$ """
    s = sigmoid(x)
    return s * (1 + x * (1 - s))

# ===== VISUALISATION =====

x = np.linspace(-4, 4, 1000)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Fonctions
ax1 = axes[0, 0]
ax1.plot(x, relu(x), 'b-', linewidth=2, label='ReLU')
ax1.plot(x, gelu(x), 'r-', linewidth=2, label='GELU')
ax1.plot(x, silu(x), 'g-', linewidth=2, label='SiLU')
ax1.axhline(0, color='k', linewidth=0.5)
ax1.axvline(0, color='k', linewidth=0.5)
ax1.set_title('Fonctions d\'activation')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Dérivées
ax2 = axes[0, 1]
ax2.plot(x, relu_derivative(x), 'b-', linewidth=2, label="ReLU'")
ax2.plot(x, gelu_derivative(x), 'r-', linewidth=2, label="GELU'")
ax2.plot(x, silu_derivative(x), 'g-', linewidth=2, label="SiLU'")
ax2.axhline(0, color='k', linewidth=0.5)
ax2.axhline(1, color='k', linewidth=0.5, linestyle='--')
ax2.set_title("Dérivées (ReLU: saut discret!)")
ax2.legend()
ax2.grid(True, alpha=0.3)

# Dérivées secondes (numériques)
eps = 1e-5
relu_d2 = (relu_derivative(x + eps) - relu_derivative(x - eps)) / (2
* eps)
gelu_d2 = (gelu_derivative(x + eps) - gelu_derivative(x - eps)) / (2
* eps)
silu_d2 = (silu_derivative(x + eps) - silu_derivative(x - eps)) / (2
* eps)

ax3 = axes[1, 0]
ax3.plot(x, relu_d2, 'b-', linewidth=2, label="ReLU''")
ax3.plot(x, gelu_d2, 'r-', linewidth=2, label="GELU''")
ax3.plot(x, silu_d2, 'g-', linewidth=2, label="SiLU''")
ax3.axhline(0, color='k', linewidth=0.5)
ax3.set_title("Dérivées secondes (ReLU: pic en x=0!)")
ax3.legend()
ax3.grid(True, alpha=0.3)
ax3.set_ylim(-1, 1)

# Zoom autour de 0
ax4 = axes[1, 1]
x_zoom = np.linspace(-0.5, 0.5, 500)
ax4.plot(x_zoom, relu(x_zoom), 'b-', linewidth=2, label='ReLU')
ax4.plot(x_zoom, gelu(x_zoom), 'r-', linewidth=2, label='GELU')
ax4.plot(x_zoom, silu(x_zoom), 'g-', linewidth=2, label='SiLU')
ax4.axhline(0, color='k', linewidth=0.5)
ax4.axvline(0, color='k', linewidth=0.5)
ax4.set_title("Zoom autour de x=0")
ax4.legend()
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('activations_comparison.png', dpi=150)
plt.show()

```

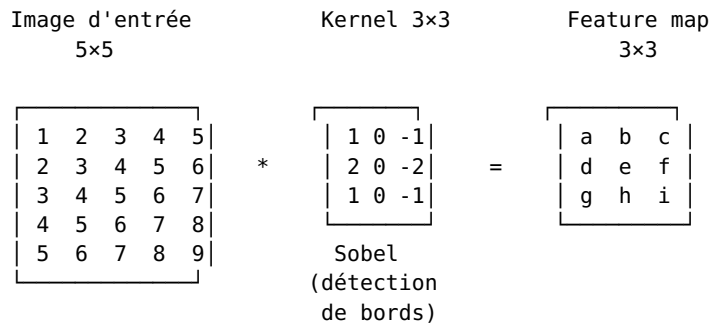
Chapitre 5 : Réseaux Convolutionnels (CNN)

5.1 Motivation

Pour les images, un MLP standard a des limitations : - **Trop de paramètres** : image $224 \times 224 \times 3 = 150,528$ entrées - **Pas d'invariance spatiale** : un chat en haut à gauche $\neq$ chat en bas à droite - **Pas d'exploitation de la structure locale**

5.2 L'Opération de Convolution

La convolution 2D applique un **filtre** (kernel) sur l'image :



Formule :

$$(I * K)[i, j] = \sum_m \sum_n I[i+m, j+n] \times K[m, n]$$

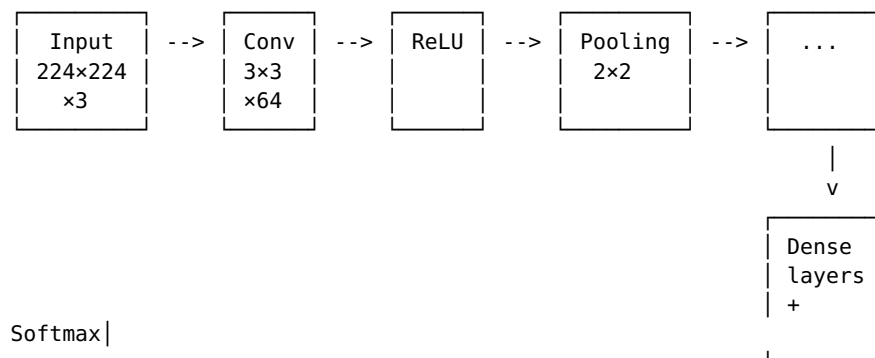
5.3 Paramètres de Convolution

Paramètre	Description
Kernel size	Taille du filtre (ex: 3x3, 5x5)
Stride	Pas de déplacement
Padding	Ajout de zéros autour de l'image
Channels	Nombre de filtres (profondeur)

Taille de sortie :

$$H_{out} = (H_{in} + 2 \times padding - kernel_size) / stride + 1$$

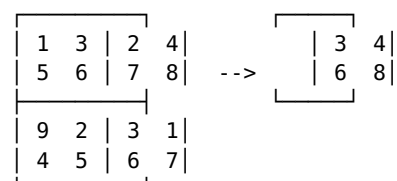
5.4 Architecture Typique



5.5 Pooling

Réduit la dimension spatiale :

Max Pooling 2x2:



5.6 Partage de Poids

Avantage clé des CNN : Le même kernel est appliqué sur toute l'image.

Nombre de paramètres:

MLP: $224 \times 224 \times 3 \times 1000 \approx 150$ millions
CNN: $3 \times 3 \times 3 \times 64 = 1,728$ (premier layer)

Réduction massive !

5.7 Code : CNN Simple

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    """
    CNN simple pour MNIST/CIFAR.
    """

    def __init__(self, num_classes=10):
        super().__init__()

        # Couches convolutionnelles
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)

        # Couches fully connected
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, num_classes)

        # Pooling
        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        # Conv1 + ReLU + Pool: 28x28 -> 14x14
        x = self.pool(F.relu(self.conv1(x)))

        # Conv2 + ReLU + Pool: 14x14 -> 7x7
        x = self.pool(F.relu(self.conv2(x)))

        # Flatten
        x = x.view(-1, 64 * 7 * 7)

        # FC layers
        x = F.relu(self.fc1(x))
        x = self.fc2(x)

        return x

# Créer le modèle
model = SimpleCNN()
print(f"Paramètres: {sum(p.numel() for p in model.parameters()):,}")
```

5.8 Implications pour l'Extraction

Les CNN ont des **défis supplémentaires** pour l'extraction :

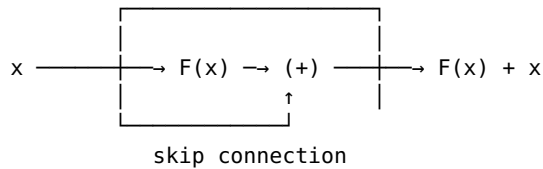
1. **Partage de poids** : Le même kernel est réutilisé → plus d'information par paramètre
2. **Structure spatiale** : L'attaquant doit comprendre où le kernel s'applique
3. **Profondeur** : Réseaux modernes très profonds (ResNet-50 = 50 couches)

Chapitre 6 : Architectures Modernes

6.1 ResNet et Skip Connections

Problème : Les réseaux très profonds souffrent de **vanishing gradient**.

Solution (He et al., 2015) : Ajouter des **connexions résiduelles** :



Formule :

$$y = F(x) + x$$

Au lieu d'apprendre $F(x) = H(x)$, on apprend $F(x) = H(x) - x$ (le "résidu").

Avantage : Le gradient peut "sauter" les couches via la connexion directe.

6.2 Transformers et Attention

Paper : "Attention Is All You Need" (Vaswani et al., 2017)

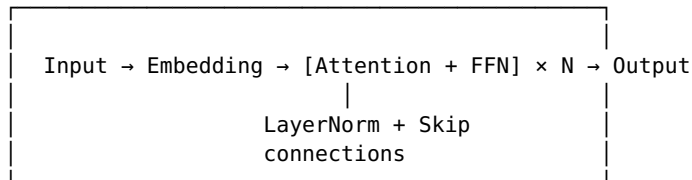
Mécanisme d'attention :

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

où:

- Q = Queries (ce qu'on cherche)
- K = Keys (ce avec quoi on compare)
- V = Values (ce qu'on retourne)
- d_k = dimension des keys

Architecture simplifiée :



Point clé : Les Transformers utilisent **GELU** (pas ReLU) !

6.3 Pourquoi GELU dans les Transformers ?

1. **Performance empirique :** GELU donne de meilleurs résultats que ReLU sur les tâches de NLP
2. **Lissité :** Évite les problèmes des gradients discontinus
3. **Interprétation probabiliste :** Pondération par la "probabilité d'être grand"

6.4 Code : Bloc Transformer Simplifié

```
import torch
import torch.nn as nn

class SelfAttention(nn.Module):
    """
    Multi-Head Self-Attention.
    """

    def __init__(self, embed_dim, num_heads):
        super().__init__()
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        self.W_q = nn.Linear(embed_dim, embed_dim)
        self.W_k = nn.Linear(embed_dim, embed_dim)
```

```

        self.W_v = nn.Linear(embed_dim, embed_dim)
        self.W_o = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        B, T, C = x.shape # batch, seq_len, channels

        # Calcul de Q, K, V
        Q = self.W_q(x)
        K = self.W_k(x)
        V = self.W_v(x)

        # Attention scores
        scores = (Q @ K.transpose(-2, -1)) / (self.head_dim ** 0.5)
        attn = torch.softmax(scores, dim=-1)

        # Output
        out = attn @ V
        out = self.W_o(out)

    return out

class TransformerBlock(nn.Module):
    """
    Un bloc Transformer avec GELU.
    """

    def __init__(self, embed_dim, num_heads, ff_dim):
        super().__init__()

        self.attention = SelfAttention(embed_dim, num_heads)
        self.ln1 = nn.LayerNorm(embed_dim)
        self.ln2 = nn.LayerNorm(embed_dim)

        # Feed-forward avec GELU !
        self.ff = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.GELU(), # <-- Pas ReLU !
            nn.Linear(ff_dim, embed_dim)
        )

    def forward(self, x):
        # Attention + skip connection
        x = x + self.attention(self.ln1(x))

        # Feed-forward + skip connection
        x = x + self.ff(self.ln2(x))

    return x

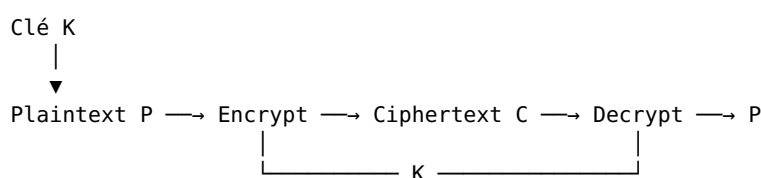
```

PARTIE II : CRYPTOGRAPHIE ET SÉCURITÉ ML

Chapitre 7 : Fondamentaux de Cryptographie

7.1 Concepts de Base

Chiffrement symétrique :



Modèles d'attaque :

Attaquant connaît Peut faire

Modèle		
Ciphertext-only	C uniquement	Observer
Known-plaintext	Paires (P, C)	Observer
Chosen-plaintext	Choisir P	Obtenir C
Chosen-ciphertext	Choisir C	Obtenir P

7.2 Cryptanalyse Différentielle

Idée (Biham & Shamir, 1990) :

Observer comment les **différences** en entrée se propagent vers la sortie.

$$\Delta P = P \oplus P' \quad (\text{différence en entrée})$$

↓

Encrypt
↓

$$\Delta C = C \oplus C' \quad (\text{différence en sortie})$$

Si ΔC est **prévisible** à partir de ΔP , on peut récupérer de l'information sur la clé.

7.3 Analogie avec les DNN

Observation clé de Carlini et al. :

Un réseau de neurones est comme un chiffrement ! - Entrée x = "plaintext" - Poids W = "clé secrète" - Sortie y = "ciphertext"

Différence cruciale :

Crypto	DNN
Conçu pour résister	Pas conçu pour la sécurité
Confusion + Diffusion	Structure mathématique exploitable
S-boxes non-linéaires	ReLU = piecewise linear !

Chapitre 8 : Modèles d'Attaque en ML

8.1 Niveaux d'Accès

MODÈLES D'ACCÈS		
WHITE-BOX	RAW-OUTPUT	HARD-LABEL
• Architecture • Poids • Gradients • Tout !	• Logits ou probas • Pas d'accès aux poids	• Classe prédite seulement • Minimum d'information
Facilité d'attaque: +++ → ++ → +		

8.2 Types d'Attaques

Attaque	Objectif	Méthode
Model Extraction	Voler les poids	Requêtes + apprentissage
Membership Inference	Savoir si x était dans le training set Reconstruire les	Analyse des sorties

Model Inversion	données d'entraînement	Optimisation
Adversarial Examples	Tromper le modèle	Perturbations calculées

8.3 Model Extraction : Approches

Approche 1 : Distillation (Tramèr et al., 2016)

1. Générer des requêtes x
2. Obtenir les réponses y du modèle cible
3. Entraîner un modèle "voleur" sur (x, y)

Approche 2 : Cryptanalytique (Carlini et al., 2020)

1. Exploiter la structure mathématique du réseau
2. Trouver les "points critiques" (pour ReLU)
3. Résoudre un système d'équations pour extraire les poids

8.4 Code : Oracle et Attaque Simple

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ModelOracle:
    """
    Simule un accès API à un modèle.
    """

    def __init__(self, model, access_level='hard_label'):
        """
        Args:
            model: Le modèle cible
            access_level: 'white_box', 'raw_output', 'hard_label'
        """
        self.model = model
        self.model.eval()
        self.access_level = access_level
        self.query_count = 0

    def query(self, x):
        """Effectue une requête au modèle."""
        self.query_count += 1

        with torch.no_grad():
            logits = self.model(x)

        if self.access_level == 'white_box':
            return {
                'logits': logits,
                'probabilities': F.softmax(logits, dim=-1),
                'prediction': logits.argmax(dim=-1)
            }
        elif self.access_level == 'raw_output':
            return F.softmax(logits, dim=-1)
        else: # hard_label
            return logits.argmax(dim=-1)

    def get_query_count(self):
        return self.query_count

class SimpleModelStealer:
    """
    Attaque de model stealing par distillation.
    """

    def __init__(self, oracle, input_dim, output_dim,
hidden_dim=64):
        self.oracle = oracle

        # Modèle voleur
```

```

        self.stolen_model = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

        self.optimizer =
torch.optim.Adam(self.stolen_model.parameters())

    def steal(self, num_queries=1000, batch_size=32):
        """
        Vole le modèle en faisant des requêtes.
        """
        for _ in range(num_queries // batch_size):
            # Générer des entrées aléatoires
            x = torch.randn(batch_size,
self.stolen_model[0].in_features)

            # Obtenir les labels du modèle cible
            with torch.no_grad():
                if self.oracle.access_level == 'hard_label':
                    labels = self.oracle.query(x)
                else:
                    labels = self.oracle.query(x).argmax(dim=-1)

            # Entraîner le modèle voleur
            self.optimizer.zero_grad()
            pred = self.stolen_model(x)
            loss = nn.CrossEntropyLoss()(pred, labels)
            loss.backward()
            self.optimizer.step()

        return self.stolen_model

    def evaluate_fidelity(self, test_x):
        """Mesure la fidélité (accord avec l'original)."""
        with torch.no_grad():
            original_pred = self.oracle.query(test_x)
            if self.oracle.access_level != 'hard_label':
                original_pred = original_pred.argmax(dim=-1)

            stolen_pred = self.stolen_model(test_x).argmax(dim=-1)

            fidelity = (original_pred == stolen_pred).float().mean()

        return fidelity.item()

```

Chapitre 9 : Model Stealing et Extraction

9.1 Historique des Attaques

Année	Paper	Contribution
2016	Tramèr et al.	Premier model stealing via API
2020	Carlini et al. (CRYPTO)	Extraction cryptanalytique ReLU
2024	Canales-Martínez et al. (EUROCRYPT)	Temps polynomial
2025	Carlini et al. (EUROCRYPT)	Hard-label setting

9.2 Différence Fondamentale

DISTILLATION	EXTRACTION CRYPTANALYTIQUE
Entraîne un nouveau modèle qui "imite"	Extrait les poids EXACTS

- Poids différents
- Poids identiques (à permutation près)
- Fidélité < 100%
- Fidélité = 100%
- Fonctionne sur tout piecewise)
- Nécessite ReLU (activations)

9.3 Métriques

Métrique	Définition
Fidélité (Fidelity)	% d'accord sur les prédictions
Agreement	Accord sur les probabilités
Task Accuracy	Précision sur la tâche originale
Query Complexity	Nombre de requêtes nécessaires

PARTIE III : ATTAQUES D'EXTRACTION CRYPTANALYTIQUES

Chapitre 10 : CRYPTO 2020 - L'Attaque Fondatrice

10.1 Paper

“Cryptanalytic Extraction of Neural Network Models” Carlini, Jagielski, Mironov CRYPTO 2020

10.2 Idée Principale

Exploiter la structure piecewise-linear de ReLU pour extraire les poids exacts.

1. Un réseau ReLU divise l'espace en régions linéaires
2. Les frontières entre régions = hyperplans définis par les poids
3. En trouvant ces frontières, on peut reconstruire les poids

10.3 Points Critiques

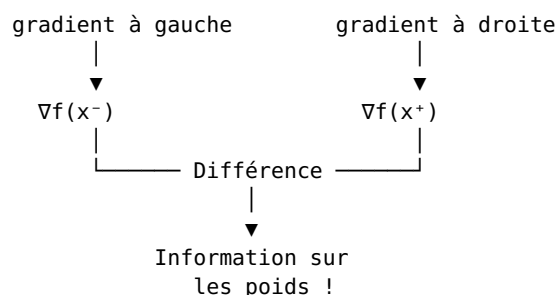
Définition : Un point critique est un point où au moins un neurone ReLU passe de actif à inactif (ou vice versa).

Point critique x^* :

Il existe un neurone j tel que $w_j^T x^* + b_j = 0$

Pourquoi c'est utile ?

Autour d'un point critique, le gradient CHANGE :



10.4 Algorithme Simplifié

ALGORITHME D'EXTRACTION CRYPTO 2020

=====

Entrée: Oracle f (accès raw-output)
Sortie: Poids W et biais b estimés

1. TROUVER LES POINTS CRITIQUES
Pour $i = 1, \dots, N$:
 - Choisir une direction aléatoire d
 - Chercher par recherche binaire un point x^* où ∇f change le long de d
 - Stocker x^* et les gradients de part et d'autre
2. REGROUPER LES POINTS PAR NEURONE
 - Les points critiques d'un même neurone sont sur le même hyperplan
 - Utiliser la géométrie pour les identifier
3. EXTRAIRE LES POIDS
 - Pour chaque groupe, l'hyperplan donne w et b
 - La différence de gradients donne le signe
4. RECONSTRUIRE LE RÉSEAU COUCHE PAR COUCHE

10.5 Complexité

Paramètre	Valeur
Requêtes	$O(n^2 \times d)$ où n = neurones, d = dimension
Précision	Exacte (à la précision numérique près)
Limitation	Temps exponentiel en la profondeur

10.6 Code : Détection de Points Critiques

```
import numpy as np
import torch
import torch.nn as nn

def numerical_gradient(oracle_fn, x, epsilon=1e-5):
    """
    Calcule le gradient par différences finies.

    Args:
        oracle_fn: fonction x -> y (scalaire ou vecteur)
        x: point où calculer le gradient
        epsilon: pas pour les différences finies

    Returns:
        gradient estimé
    """
    x = np.array(x, dtype=np.float64)
    grad = np.zeros_like(x)

    for i in range(len(x)):
        x_plus = x.copy()
        x_minus = x.copy()
        x_plus[i] += epsilon
        x_minus[i] -= epsilon

        f_plus = oracle_fn(x_plus)
        f_minus = oracle_fn(x_minus)

        grad[i] = (f_plus - f_minus) / (2 * epsilon)

    return grad

def find_critical_point_binary_search(oracle_fn, x_start, direction,
                                     t_min=-5, t_max=5,
                                     tolerance=1e-6):
    """
    Trouve un point critique par recherche binaire.

    Un point critique est détecté quand le gradient change
    significativement le long de la direction.
```

```

    Args:
        oracle_fn: fonction à analyser
        x_start: point de départ
        direction: direction de recherche
        t_min, t_max: intervalle de recherche
        tolerance: précision

    Returns:
        point critique ou None
    """
    direction = direction / np.linalg.norm(direction)

    # Gradient aux extrémités
    x_low = x_start + t_min * direction
    x_high = x_start + t_max * direction

    grad_low = numerical_gradient(oracle_fn, x_low)
    grad_high = numerical_gradient(oracle_fn, x_high)

    # S'il n'y a pas de changement significatif, pas de point
    critique
    if np.allclose(grad_low, grad_high, atol=1e-3):
        return None

    # Recherche binaire
    while t_max - t_min > tolerance:
        t_mid = (t_min + t_max) / 2
        x_mid = x_start + t_mid * direction
        grad_mid = numerical_gradient(oracle_fn, x_mid)

        # Décider de quel côté chercher
        if not np.allclose(grad_low, grad_mid, atol=1e-3):
            t_max = t_mid
            grad_high = grad_mid
        else:
            t_min = t_mid
            grad_low = grad_mid

    return x_start + (t_min + t_max) / 2 * direction

def extract_critical_points(oracle_fn, input_dim, num_samples=100):
    """
    Trouve plusieurs points critiques du réseau.

    Returns:
        Liste de (point_critique, gradient_avant, gradient_après)
    """
    critical_points = []

    for _ in range(num_samples):
        # Point de départ aléatoire
        x_start = np.random.randn(input_dim) * 2

        # Direction aléatoire
        direction = np.random.randn(input_dim)

        # Chercher un point critique
        cp = find_critical_point_binary_search(oracle_fn, x_start,
        direction)

        if cp is not None:
            # Calculer les gradients de part et d'autre
            eps = 1e-4 * direction / np.linalg.norm(direction)
            grad_before = numerical_gradient(oracle_fn, cp - eps)
            grad_after = numerical_gradient(oracle_fn, cp + eps)

            critical_points.append((cp, grad_before, grad_after))

    return critical_points

# ===== EXEMPLE =====

```

```
# Créer un réseau ReLU cible
target_model = nn.Sequential(
    nn.Linear(2, 3),
    nn.ReLU(),
    nn.Linear(3, 1)
)

def oracle(x):
    with torch.no_grad():
        x_tensor = torch.tensor(x, dtype=torch.float32).unsqueeze(0)
        return target_model(x_tensor).item()

# Trouver les points critiques
critical_pts = extract_critical_points(oracle, input_dim=2,
num_samples=50)
print(f"Points critiques trouvés: {len(critical_pts)}")

# Analyser les différences de gradients
for i, (cp, grad_before, grad_after) in enumerate(critical_pts[:5]):
    diff = grad_after - grad_before
    print(f"\nPoint {i+1}:")
    print(f"  Position: {cp}")
    print(f"  Δgradient: {diff}")
```

Chapitre 11 : EUROCRYPT 2024 - Temps Polynomial

11.1 Paper

“Polynomial Time Cryptanalytic Extraction of Neural Network Models” Canales-Martínez, Fomin, Makarevič, et al. EUROCRYPT 2024

11.2 Amélioration Principale

Problème de CRYPTO 2020 : Temps exponentiel en la profondeur.

Solution : Algorithme polynomial grâce à : 1. Analyse plus fine des points critiques 2. Techniques algébriques avancées 3. Traitement couche par couche optimisé

11.3 Complexité Améliorée

Aspect	CRYPTO 2020	EUROCRYPT 2024
Temps	Exponentiel en L	Polynomial
Requêtes	$O(n^2 \times d)$	$O(n^2 \times d)$
Applicable à	Réseaux peu profonds	Réseaux profonds

11.4 Techniques Clés

- Differential equations on activation boundaries
- Polynomial system solving
- Layerwise extraction with propagation

Chapitre 12 : EUROCRYPT 2025 - Hard-Label Setting

12.1 Paper

“Polynomial Time Cryptanalytic Extraction in Hard-Label Setting” Carlini et al. EUROCRYPT 2025

12.2 Défi

Hard-label : L'attaquant n'a accès qu'à la **classe prédite**, pas aux probabilités !

Raw-output: $x \longrightarrow [0.1, 0.7, 0.2]$ (probabilités)
Hard-label: $x \longrightarrow 1$ (juste la classe)

C'est beaucoup moins d'information !

12.3 Approche

Idee : Utiliser les **frontières de décision** comme source d'information.

1. Trouver deux points x_1, x_2 de classes différentes
2. Par recherche binaire, trouver un point x^* sur la frontière de décision
3. La frontière révèle de l'information sur les poids

12.4 Complexité

Aspect	Hard-label
Requêtes	Plus nombreuses (facteur log)
Précision	Légèrement réduite
Applicable	Réseaux ReLU

PARTIE IV : AU-DELÀ DE ReLU - LE CŒUR DE LA THÈSE

Chapitre 13 : Pourquoi GELU/SiLU Résistant

13.1 Le Problème Fondamental

Les attaques CRYPTO/EUROCRYPT reposent sur la détection de points critiques.

Pour ReLU :

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x < 0 \end{cases}$$

→ Changement DISCRET → Détectable !

Pour GELU :

$$\text{GELU}'(x) = \Phi(x) + x \cdot \phi(x)$$

→ Variation CONTINUE → Pas de signature claire !

13.2 Analyse Mathématique

Dérivée seconde :

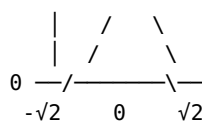
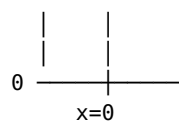
$$\text{ReLU}''(x) = \delta(x) \quad (\text{distribution de Dirac en } 0) \\ \rightarrow \text{"Pic" détectable}$$

$$\text{GELU}''(x) = \phi(x)(2 - x^2) \\ \rightarrow \text{Fonction continue, jamais infinie}$$

Visualisation :

$$\text{ReLU}''$$

$$\text{GELU}''$$



13.3 Conséquences pour l'Extraction

Propriété	ReLU	GELU/SiLU
Points critiques	OUI, détectables	NON, pas de discontinuité
Régions linéaires	OUI	NON (non-linéaire partout)
Attaque CRYPTO 2020	Fonctionne	NE FONCTIONNE PAS

13.4 Code : Démonstration

```
import numpy as np
import torch
import torch.nn as nn

def detect_gradient_changes(oracle_fn, x_range=(-3, 3),
resolution=1000,
threshold=0.1):
    """
    Détecte les changements brusques de gradient.

    Pour ReLU: beaucoup de changements brusques
    Pour GELU: changements progressifs
    """
    x = np.linspace(x_range[0], x_range[1], resolution)
    epsilon = (x_range[1] - x_range[0]) / resolution

    # Calculer les dérivées numériques
    derivatives = []
    for xi in x:
        df = (oracle_fn(xi + epsilon) - oracle_fn(xi - epsilon)) /
(2 * epsilon)
        derivatives.append(df)

    derivatives = np.array(derivatives)

    # Détecter les changements brusques
    changes = np.abs(np.diff(derivatives))
    critical_indices = np.where(changes > threshold)[0]

    return len(critical_indices), changes

# Créer deux réseaux identiques
relu_net = nn.Sequential(
    nn.Linear(1, 5),
    nn.ReLU(),
    nn.Linear(5, 1)
)

gelu_net = nn.Sequential(
    nn.Linear(1, 5),
    nn.GELU(),
    nn.Linear(5, 1)
)

# Copier les mêmes poids
with torch.no_grad():
    gelu_net[0].weight.copy_(relu_net[0].weight)
    gelu_net[0].bias.copy_(relu_net[0].bias)
    gelu_net[2].weight.copy_(relu_net[2].weight)
    gelu_net[2].bias.copy_(relu_net[2].bias)

def relu_oracle(x):
    with torch.no_grad():
        return relu_net(torch.tensor([[x]]),
```

```
dtype=torch.float32)).item()

def gelu_oracle(x):
    with torch.no_grad():
        return gelu_net(torch.tensor([[x]]),
dtype=torch.float32)).item()

# Détecter les changements
relu_critical, _ = detect_gradient_changes(relu_oracle,
threshold=0.3)
gelu_critical, _ = detect_gradient_changes(gelu_oracle,
threshold=0.3)

print(f"Points critiques détectés:")
print(f"  ReLU: {relu_critical}")
print(f"  GELU: {gelu_critical}")
print(f"\n→ ReLU révèle sa structure, GELU la cache !")
```

Chapitre 14 : Pistes de Recherche

14.1 Question Centrale de ta Thèse

Comment extraire les poids d'un réseau utilisant GELU/SiLU ?

14.2 Piste 1 : Approximation Piecewise-Linear

Idée : Approximer GELU par une fonction piecewise-linear, puis appliquer les méthodes existantes.

$$\text{GELU}(x) \approx \text{PWL}(x) = \sum_i (a_i x + b_i) \times \mathbb{1}[x \in R_i]$$

Erreur d'approximation $\varepsilon \rightarrow$ erreur sur les poids extraits

Questions ouvertes : - Quelle précision d'approximation est nécessaire ? - L'erreur se propage-t-elle de manière contrôlée ?

14.3 Piste 2 : Points d'Inflexion

Idée : Utiliser les **points d'inflexion** (où $f''(x) = 0$) au lieu des points critiques.

Pour GELU :

$$\text{GELU}''(x) = 0 \iff x^2 = 2 \iff x = \pm\sqrt{2}$$

Ces points pourraient révéler de l'information ?

14.4 Piste 3 : Méthodes d'Optimisation

Idée : Formuler l'extraction comme un problème d'optimisation.

$$\min_{\{W, b\}} ||f_{\theta}(X) - f_{\{W,b\}}(X)||^2$$

où:

- f_{θ} = réseau cible (oracle)
- $f_{\{W,b\}}$ = réseau à optimiser
- X = ensemble de requêtes

Défis : - Paysage non-convexe - Symétries (permutation des neurones)
- Minima locaux

14.5 Piste 4 : Attaques Hybrides

Idée : Combiner extraction cryptanalytique + distillation.

1. Extraire une approximation par distillation
2. Raffiner avec des techniques cryptanalytiques
3. Utiliser des informations side-channel

14.6 Piste 5 : Nouvelles Signatures

Question : Y a-t-il d'autres "signatures" détectables pour GELU/SiLU ?

Candidats : - Comportement asymptotique - Moments statistiques des sorties - Réponse à des entrées spécifiques

14.7 Code : Exploration des Points d'Inflexion

```
import numpy as np
from scipy.special import erf
from scipy.optimize import brentq

def gelu(x):
    return 0.5 * x * (1 + erf(x / np.sqrt(2)))

def gelu_second_derivative(x):
    """GELU''(x) =  $\phi(x)(2 - x^2)$ """
    phi = np.exp(-0.5 * x**2) / np.sqrt(2 * np.pi)
    return phi * (2 - x**2)

# Trouver les points d'inflexion
# GELU''(x) = 0 quand  $x^2 = 2$ , donc  $x = \pm\sqrt{2}$ 

inflection_points = [-np.sqrt(2), np.sqrt(2)]
print(f"Points d'inflexion de GELU: {inflection_points}")

# Pour un réseau GELU, les points d'inflexion des neurones
# sont à  $z = \pm\sqrt{2}$ , donc à  $x$  tel que  $w \cdot x + b = \pm\sqrt{2}$ 

# Cela donne:  $w \cdot x + b = \sqrt{2}$  ou  $w \cdot x + b = -\sqrt{2}$ 
# Ce sont des hyperplans parallèles !

print("\nPour extraire les poids, on pourrait chercher les hyperplans")
print(" $w \cdot x + b = \pm\sqrt{2}$  au lieu des hyperplans  $w \cdot x + b = 0$  de ReLU")
```

Chapitre 15 : Défenses et Contre-mesures

15.1 Types de Défenses

DÉFENSES		
DÉTECTION	PERTURBATION	WATERMARKING
Détecter les attaques	Ajouter du bruit aux réponses	Marquer le modèle pour prouver la propriété

15.2 Perturbation des Sorties

Idée : Ajouter du bruit aux logits/probabilités.

```
def noisy_prediction(model, x, noise_std=0.1):
    logits = model(x)
    noisy_logits = logits + torch.randn_like(logits) * noise_std
    return F.softmax(noisy_logits, dim=-1)
```

Trade-off : - Plus de bruit → plus difficile à extraire - Plus de bruit → moins précis pour l'utilisateur légitime

15.3 Rate Limiting

Idée : Limiter le nombre de requêtes.

```
class RateLimitedOracle:
    def __init__(self, model, max_queries=10000):
```

```

self.model = model
self.max_queries = max_queries
self.query_count = 0

def query(self, x):
    if self.query_count >= self.max_queries:
        raise Exception("Rate limit exceeded")
    self.query_count += 1
    return self.model(x)

```

15.4 Watermarking

Idée : Incorporer une "signature" dans le modèle.

1. Choisir des entrées "trigger" spéciales
2. Entraîner le modèle à donner des réponses spécifiques sur ces triggers
3. Si un modèle volé a les mêmes réponses sur les triggers → preuve de vol

15.5 MEA-Defender (Défense Avancée)

Paper : "Watermarking Deep Neural Networks with Model Extraction Attacks"

Idée : Créer des watermarks qui ressemblent aux données normales.

15.6 Code : Défenses Simples

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class DefendedModel(nn.Module):
    """
    Modèle avec plusieurs défenses.
    """

    def __init__(self, base_model, noise_std=0.1, temperature=1.0):
        super().__init__()
        self.base_model = base_model
        self.noise_std = noise_std
        self.temperature = temperature
        self.query_count = 0

    def forward(self, x, add_noise=True):
        self.query_count += 1

        logits = self.base_model(x)

        # 1. Ajouter du bruit
        if add_noise and self.training == False:
            logits = logits + torch.randn_like(logits) *

self.noise_std

        # 2. Temperature scaling (aplatit les probabilités)
        probs = F.softmax(logits / self.temperature, dim=-1)

        return probs

    def predict(self, x):
        """Retourne uniquement la classe (hard-label)."""
        return self.forward(x).argmax(dim=-1)

# Tester l'effet sur l'extraction
base_model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
)

# Sans défense

```

```

oracle_normal = base_model

# Avec défense
oracle_defended = DefendedModel(base_model, noise_std=0.5,
temperature=2.0)

# L'attaquant aura plus de mal avec oracle_defended

```

PARTIE V : EXERCICES ET CODE

Exercice Complet 1 : Perceptron et MLP

```

"""
EXERCICE 1: Implémenter un perceptron et un MLP from scratch.
"""

import numpy as np

# ===== PERCEPTRON =====

class Perceptron:
    def __init__(self, n_features):
        self.weights = np.random.randn(n_features) * 0.01
        self.bias = 0.0

    def predict(self, x):
        return 1 if np.dot(self.weights, x) + self.bias >= 0 else 0

    def train(self, X, y, lr=0.1, epochs=100):
        for _ in range(epochs):
            for xi, yi in zip(X, y):
                pred = self.predict(xi)
                error = yi - pred
                self.weights += lr * error * xi
                self.bias += lr * error

# Test AND
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1])

p = Perceptron(2)
p.train(X, y)
print("AND:", [p.predict(xi) for xi in X])

# ===== MLP =====

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

class MLP:
    def __init__(self, sizes):
        self.W = [np.random.randn(sizes[i+1], sizes[i]) *
np.sqrt(2/sizes[i])
                    for i in range(len(sizes)-1)]
        self.b = [np.zeros((sizes[i+1], 1)) for i in
range(len(sizes)-1)]

    def forward(self, x):
        self.a = [x]
        self.z = []
        for W, b in zip(self.W, self.b):
            z = W @ self.a[-1] + b
            self.z.append(z)
            self.a.append(sigmoid(z))
        return self.a[-1]

    def backward(self, y, lr):
        m = y.shape[1]
        delta = (self.a[-1] - y) * self.a[-1] * (1 - self.a[-1])

```

```

        for l in range(len(self.W) - 1, -1, -1):
            dW = (1/m) * delta @ self.a[l].T
            db = (1/m) * np.sum(delta, axis=1, keepdims=True)

            if l > 0:
                delta = (self.W[l].T @ delta) * self.a[l] * (1 -
self.a[l])

            self.W[l] -= lr * dW
            self.b[l] -= lr * db

    def train(self, X, y, lr=1.0, epochs=10000):
        for _ in range(epochs):
            self.forward(X)
            self.backward(y, lr)

# Test XOR
X = np.array([[0,0,1,1], [0,1,0,1]])
y = np.array([[0,1,1,0]])

mlp = MLP([2, 4, 1])
mlp.train(X, y, lr=2.0, epochs=10000)
print("XOR:", mlp.forward(X).round(2))

```

Exercice Complet 2 : Fonctions d'Activation

```

"""
EXERCICE 2: Implémenter et visualiser les fonctions d'activation.
"""

```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erf

# Définitions
def relu(x): return np.maximum(0, x)
def relu_d(x): return (x > 0).astype(float)

def gelu(x): return 0.5 * x * (1 + erf(x / np.sqrt(2)))
def gelu_d(x):
    phi_cdf = 0.5 * (1 + erf(x / np.sqrt(2)))
    phi_pdf = np.exp(-0.5 * x**2) / np.sqrt(2 * np.pi)
    return phi_cdf + x * phi_pdf

def silu(x): return x / (1 + np.exp(-x))
def silu_d(x):
    s = 1 / (1 + np.exp(-x))
    return s * (1 + x * (1 - s))

# Visualisation
x = np.linspace(-4, 4, 500)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Fonctions
axes[0].plot(x, relu(x), 'b-', lw=2, label='ReLU')
axes[0].plot(x, gelu(x), 'r-', lw=2, label='GELU')
axes[0].plot(x, silu(x), 'g-', lw=2, label='SiLU')
axes[0].axhline(0, color='k', lw=0.5)
axes[0].legend()
axes[0].set_title('Fonctions d\'activation')
axes[0].grid(True, alpha=0.3)

# Dérivées
axes[1].plot(x, relu_d(x), 'b-', lw=2, label="ReLU'")
axes[1].plot(x, gelu_d(x), 'r-', lw=2, label="GELU'")
axes[1].plot(x, silu_d(x), 'g-', lw=2, label="SiLU'")
axes[1].axhline(0, color='k', lw=0.5)
axes[1].axhline(1, color='k', lw=0.5, ls='--')
axes[1].legend()
axes[1].set_title('Dérivées')

```

```

axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('activations.png', dpi=150)
print("Figure sauvegardée: activations.png")

```

Exercice Complet 3 : Détection de Points Critiques

```

"""
EXERCICE 3: Détecter les points critiques d'un réseau ReLU.
"""

import numpy as np
import torch
import torch.nn as nn

# Créer un réseau ReLU
model = nn.Sequential(
    nn.Linear(2, 4),
    nn.ReLU(),
    nn.Linear(4, 1)
)

def oracle(x):
    with torch.no_grad():
        return model(torch.tensor(x, dtype=torch.float32)).numpy()

def numerical_gradient(f, x, eps=1e-5):
    grad = np.zeros_like(x)
    for i in range(len(x)):
        e = np.zeros_like(x)
        e[i] = eps
        grad[i] = (f(x + e) - f(x - e)) / (2 * eps)
    return grad.flatten()

def find_critical_points(f, n_samples=100):
    critical = []
    for _ in range(n_samples):
        x0 = np.random.randn(2) * 2
        d = np.random.randn(2)
        d = d / np.linalg.norm(d)

        # Recherche binaire
        t_low, t_high = -3, 3
        g_low = numerical_gradient(f, x0 + t_low * d)

        for _ in range(50):
            t_mid = (t_low + t_high) / 2
            g_mid = numerical_gradient(f, x0 + t_mid * d)

            if not np.allclose(g_low, g_mid, atol=0.1):
                t_high = t_mid
            else:
                t_low = t_mid
                g_low = g_mid

            if t_high - t_low < 1e-5:
                break

        cp = x0 + (t_low + t_high) / 2 * d
        critical.append(cp)

    return critical

# Trouver les points critiques
cps = find_critical_points(oracle, n_samples=50)
print(f"Points critiques trouvés: {len(cps)}")

# Ces points révèlent les hyperplans des neurones ReLU !

```

Exercice Complet 4 : Attaque de Model Stealing

```
"""
EXERCICE 4: Implémenter une attaque de model stealing simple.
"""

import torch
import torch.nn as nn
import torch.optim as optim

# Modèle cible (secret)
target_model = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 10)
)

# Oracle
def oracle(x):
    with torch.no_grad():
        return target_model(x).argmax(dim=-1)

# Modèle voleur
stolen_model = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 10)
)

optimizer = optim.Adam(stolen_model.parameters())

# Attaque
for step in range(1000):
    x = torch.randn(64, 10)
    labels = oracle(x)

    optimizer.zero_grad()
    loss = nn.CrossEntropyLoss()(stolen_model(x), labels)
    loss.backward()
    optimizer.step()

    if step % 200 == 0:
        print(f"Step {step}, Loss: {loss.item():.4f}")

# Évaluer la fidélité
test_x = torch.randn(1000, 10)
target_pred = oracle(test_x)
stolen_pred = stolen_model(test_x).argmax(dim=-1)
fidelity = (target_pred == stolen_pred).float().mean()
print(f"\nFidélité: {fidelity.item():.2%}")
```

RÉFÉRENCES BIBLIOGRAPHIQUES

Papers Fondamentaux

1. **Carlini, N., Jagielski, M., & Mironov, I.** (2020). *Cryptanalytic Extraction of Neural Network Models*. CRYPTO 2020.
2. **Canales-Martínez, B., et al.** (2024). *Polynomial Time Cryptanalytic Extraction of Neural Network Models*. EUROCRYPT 2024.
3. **Carlini, N., et al.** (2025). *Polynomial Time Cryptanalytic Extraction in Hard-Label Setting*. EUROCRYPT 2025.
4. **Hendrycks, D., & Gimpel, K.** (2016). *Gaussian Error Linear Units (GELUs)*. arXiv:1606.08415.

5. **Ramachandran, P., Zoph, B., & Le, Q.** (2017). *Searching for Activation Functions*. arXiv:1710.05941.
6. **Tramèr, F., et al.** (2016). *Stealing Machine Learning Models through Prediction APIs*. USENIX Security.

Livres

- **Goodfellow, I., Bengio, Y., & Courville, A.** (2016). *Deep Learning*. MIT Press.
- **Bishop, C.** (2006). *Pattern Recognition and Machine Learning*. Springer.

Ressources en Ligne

- Deep Learning Book: <https://www.deeplearningbook.org/>
 - Neural Networks: Zero to Hero: <https://karpathy.ai/zero-to-hero.html>
 - 3Blue1Brown Neural Networks: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
-

CONCLUSION

Ce cours complet couvre tous les aspects nécessaires pour ta thèse sur les attaques d'extraction de DNN au-delà de ReLU :

1. **Fondamentaux DL** : Comprendre comment les réseaux fonctionnent
2. **Cryptographie** : L'analogie qui motive les attaques
3. **Attaques existantes** : Ce qui fonctionne sur ReLU
4. **Le défi central** : Pourquoi GELU/SiLU résistent
5. **Pistes de recherche** : Comment avancer

Ta contribution potentielle : - Adapter les attaques cryptanalytiques aux activations lisses - Proposer de nouvelles défenses - Comprendre les limites théoriques de l'extraction

Bonne thèse !

Document créé le 31 Janvier 2026 Thèse de Tidiane DIALLO - EPT/CRISIN'2D